Design Justification Report

Memory-Mapped INT8 MAC Accelerator for GEMM-Style Workloads

Roshan Bernard Premarajan
ECE 410/510 Hardware for Artificial Intelligence and Machine Learning
Spring 2026

1. Problem and Motivation

This project targets the multiply-accumulate operation that dominates tiled GEMM workloads used in machine learning inference. The original software benchmark uses a 16 x 16 x 16 tiled GEMM with tile size 4. The workload performs 8192 FLOP per run and the measured median Python software runtime is 1931.06 us, giving a throughput of 0.004242 GOPS.

The motivation for custom hardware is that GEMM spends most of its work in repeated multiply-accumulate operations. Even a small hardware MAC block can execute this core operation more directly than scalar Python software. The final M4 design therefore implements the core signed INT8 MAC operation and exposes it through a simple memory-mapped interface.

The submitted hardware is not a full systolic array or full tiled GEMM engine. It is a memory-mapped signed INT8 MAC accelerator that represents the dominant compute primitive inside GEMM.

2. Roofline Analysis

The target kernel has moderate arithmetic intensity because each MAC performs useful arithmetic while consuming compact INT8 operands. In the M4 model, one MAC is counted as two operations: one multiply and one accumulate.

The final roofline plot is shown in Figure 1 and committed as `project/m4/bench/roofline_final.png`. The software baseline point uses the measured Python tiled GEMM throughput of 0.004242 GOPS. The M4 accelerator point uses the timing-derived throughput of 0.1584 GOPS.

The target OpenLane clock period was 10 ns, but the worst setup slack was -2.624 ns. Therefore, the corrected timing-derived period is 12.624 ns, corresponding to 79.2 MHz. With one MAC per cycle and two operations per MAC, the accelerator throughput is 0.1584 GOPS.

The design is compute-limited in the local MAC datapath model, but the complete memory-mapped system would become interface-limited if the host must write operands for every MAC. This means the roofline point is best interpreted as a compute-core throughput point, not as a fully measured end-to-end GEMM accelerator point.

3. Precision and Data Format

The design uses signed INT8 operands and a signed 32-bit accumulator. INT8 was selected because many inference workloads use reduced precision to reduce memory bandwidth, area, and switching energy compared with FP32. The 32-bit accumulator prevents immediate overflow for small dot products and follows the common quantized inference pattern of INT8 multiply with wider accumulation.

The RTL implements signed multiplication using two 8-bit signed inputs. The 16-bit product is sign-extended into the 32-bit accumulator. This preserves signed behavior for positive and negative test values.

4. Dataflow and Architecture

The final architecture is a simple host-driven MAC dataflow. The host writes operands and control information through the memory-mapped interface. The interface drives the compute core inputs. When the valid signal is asserted, the compute core multiplies the signed INT8 operands and accumulates the result into a signed 32-bit output register.

The design does not implement a systolic array, local SRAM tile buffer, or multi-PE dataflow. The practical dataflow is therefore no-local-reuse at the hardware level. Reuse is managed by the host/testbench sequence, not by an internal hardware memory hierarchy.

The main RTL modules are:

- `top.sv`: top-level integration
- `interface_ctrl.sv`: memory-mapped interface
- `compute_core.sv`: signed INT8 MAC datapath

5. Hardware Interface

The hardware interface is a lightweight memory-mapped interface. It includes host request valid, ready, write enable, address, write data, and read data signals. The interface writes operands and control registers and returns the accumulated result.

This interface is simple and easy to verify, but it is not optimized for sustained GEMM throughput. Since operands are written through registers, the complete system can become interface-bound if every MAC requires separate host transactions. A streaming interface, DMA engine, or local tile buffer would be needed to approach sustained GEMM throughput.

6. Verification

The final testbench is committed as `project/m4/tb/tb_top.sv`. The final simulation log is committed as `project/m4/sim/final_run.log`. The log reports:

PASS: Full 16x16 GEMM-style host-interface-compute-host test passed

This verifies the end-to-end path from host writes, through the memory-mapped interface, into the MAC compute core, and back to host readback. The waveform image is committed as `project/m4/sim/final_waveform.png`.

The verification focuses on functional correctness of the integrated interface and compute path. It does not prove timing closure or full physical correctness; those are addressed separately through synthesis and STA reports.

7. Synthesis Results

The design was synthesized using OpenLane 2. The final configuration is committed as `project/m4/synth/config.json`, and the run log is committed as `project/m4/synth/openlane_run.log`.

The area report shows:

- Number of cells: 774
- Chip area for module `top`: 8479.382400 um^2

The power report shows:

- Internal power: 5.559857e-04 W
- Switching power: 1.590537e-04 W
- Leakage power: 4.945864e-09 W
- Total power: 7.150443e-04 W, or 0.715 mW

The timing report shows that the design completed the OpenLane flow but did not fully meet timing at the requested 10 ns clock across all corners. The worst reported setup slack is -2.624074 ns at the max_ss_100C_1v60 corner, with TNS = -32.354795 ns. The nominal TT corner did not show negative setup slack, but the slow corners did.

The most likely timing fix is to pipeline the MAC datapath by separating multiplication and accumulation across cycles.

8. Benchmark Results

The software baseline is the measured Python tiled GEMM benchmark. The median runtime is 1931.06 us for 8192 FLOP, giving 0.004242 GOPS.

The M4 accelerator throughput is derived from the corrected timing period and RTL cycle model:

Corrected period = 10 ns + 2.624 ns = 12.624 ns
Corrected frequency = 79.2 MHz
Useful work per cycle = 2 FLOP/cycle
Accelerator throughput = 0.1584 GOPS

The resulting speedup is:

Speedup = 0.1584 GOPS / 0.004242 GOPS = 37.3x

Equivalently, the accelerator time for 8192 FLOP is 51.72 us, and:

Speedup = 1931.06 us / 51.72 us = 37.3x

The accelerator energy estimate uses the synthesis power estimate:

Energy = 0.715 mW * 51.72 us = 0.03698 uJ

This is not measured silicon or FPGA energy. No measured software energy number is available.

9. What Did Not Work

The main limitation is that the final RTL implements a single memory-mapped MAC engine, not a full tiled GEMM accelerator. This was useful for completing a synthesizable and verifiable design, but it does not include local buffering, DMA, a systolic array, or multiple processing elements.

Timing also did not fully close at the requested 10 ns target across all corners. The worst setup slack was -2.624074 ns. The likely cause is the unpipelined multiply-accumulate datapath. A better version would pipeline the multiplier and accumulator or relax the clock period and re-run STA.

The benchmark is also not a true end-to-end hardware GEMM measurement. The accelerator number assumes one useful MAC per cycle and no interface stalls. In a real system, the register-based interface would reduce throughput unless operands were streamed or buffered locally.

If this project were continued, the next improvements would be adding a pipelined MAC, cycle counters, local tile storage, and a streaming or DMA-style interface. That would allow the design to report measured end-to-end cycles rather than a timing-derived throughput model.

Figures

Figure 1: Final roofline plot, committed at `project/m4/report/figures/roofline_final.png`.

Additional Implementation Details

The final M4 design was intentionally kept small so that the full path from RTL to simulation and synthesis could be completed and documented. The compute core is centered around a single signed INT8 multiplier and a signed 32-bit accumulator. This choice matches the basic operation required in GEMM, where each output element is formed by repeatedly multiplying one value from matrix A with one value from matrix B and accumulating the product.

The memory-mapped interface was selected because it gives a clear software-visible control path. The host can write input operands, trigger computation,

and read the accumulated result. This made the M3 and M4 verification easier because the testbench can behave like a simple processor or driver. The drawback is that this interface is not high bandwidth. For a real GEMM accelerator, the interface would need to feed many operands continuously. In the current design, each MAC operation depends on host-controlled register transactions, so interface overhead can dominate if the design is used as a complete GEMM engine.

The top-level module is mostly glue logic. It connects the interface controller to the compute core without extra buffering, FIFOs, or clock-domain crossing logic. This keeps the design easy to synthesize and debug. It also means the design uses only one clock domain and avoids CDC problems. Since this was the final course milestone, this was a reasonable tradeoff: the submitted hardware is coherent, synthesizable, and verifiable, even though it is not yet optimized for full accelerator throughput.

Relationship to the Original GEMM Goal

The original project direction was an INT8 tiled matrix multiplication accelerator. The final implementation should be viewed as the first hardware building block toward that goal, not as the complete tiled GEMM engine. The implemented MAC datapath is the main arithmetic primitive needed by GEMM. A complete tiled accelerator would replicate this block or arrange many copies in a systolic or SIMD structure, add local storage for tiles, and include a higher-bandwidth interface.

This distinction is important because the M4 report must match the RTL exactly. The RTL does not contain multiple processing elements, a systolic array, scratchpad memories, or a full GEMM controller. Therefore, the final report describes the submitted design as a memory-mapped INT8 MAC accelerator for GEMM-style workloads. The testbench can apply a GEMM-style sequence through the host interface, but the hardware itself remains a single MAC engine.

Timing Interpretation

The OpenLane timing result is one of the most important outcomes of this milestone. The design completed the flow, but it did not close timing at the requested 10 ns period across all corners. The worst reported setup slack was -2.624074 ns in the slow-slow 100C 1.60 V corner. This means the critical path is too long for the requested 100 MHz target under worst-case conditions.

For benchmarking, the corrected period was computed by adding the magnitude of the worst negative slack to the requested period. This gives 12.624 ns, or about 79.2 MHz. This is a conservative timing-derived estimate, not the same as a confirmed re-run at a relaxed clock period. A stronger final design would re-run OpenLane using the corrected period and confirm that setup and hold timing both pass at that new constraint.

The likely source of the timing problem is the unpipelined MAC datapath.

The design multiplies the INT8 operands and accumulates the sign-extended product into a 32-bit register. Even though the operands are small, the combined multiply and accumulate path can still become the critical path after synthesis and place-and-route. A two-stage implementation would place a register after the multiplication stage and perform accumulation in the next cycle. This would reduce the critical path delay, but it would also add latency and require small control changes.

Area and Power Interpretation

The synthesized design contains 774 cells and the reported chip area for the top module is 8479.382400 um^2. Since the design is small, most of the area is expected to come from the arithmetic datapath, interface registers, control logic, and synthesized standard cells used for the signed multiplier and accumulator. The design does not include SRAMs or large replicated compute arrays, so the area is consistent with a compact MAC-based accelerator block.

The total estimated power is 0.715 mW. Most of this is internal and switching power, while leakage is very small. The report shows internal power of 5.559857e-04 W, switching power of 1.590537e-04 W, and leakage power of 4.945864e-09 W. This is expected for a small digital block where dynamic activity dominates the power estimate.

The energy estimate in the benchmark section uses the synthesis power multiplied by the timing-derived runtime. This is useful for a first-order comparison, but it should not be treated as a measured chip result. The power estimate depends on synthesis assumptions and switching activity assumptions from the flow. A better future version would use realistic switching activity from a VCD/SAIF generated by the final GEMM-style testbench.

Verification Coverage and Limitations

The final simulation log reports that the full 16x16 GEMM-style host-interface-compute-host test passed. This is valuable because it checks more than just the isolated MAC. It confirms that the top-level interface, operand writes, compute triggering, accumulation behavior, and result readback work together.

However, this verification is still functional simulation. It does not prove all possible input combinations, all overflow cases, or all timing behavior after place-and-route. The signed INT8 datapath should ideally be tested with positive values, negative values, mixed signs, reset behavior, repeated accumulation, and valid-low hold behavior. Some of these were already covered in earlier M2 work, while M4 focuses on the integrated top-level path.

For a production-style accelerator, additional verification would be needed. This would include randomized tests, comparison against a Python golden model, assertions for interface protocol behavior, and coverage collection. Formal checks could also be added for reset behavior and register read/write consistency.

Final Assessment

The final M4 package demonstrates a complete small accelerator implementation flow: software baseline, RTL implementation, top-level integration, simulation, synthesis, benchmark calculation, roofline positioning, and design justification. The main strength is that the submitted design is coherent and traceable. The source code, simulation log, synthesis reports, benchmark files, and report all describe the same hardware.

The main weakness is that the hardware is still a first-stage MAC accelerator rather than a full GEMM accelerator. The performance number is also timing-derived rather than measured end-to-end on FPGA or silicon. These limitations are stated directly so that the final examination discussion is aligned with the actual submitted design.